Strategy-Selection Steering for Reasoning Models

Avni Badiwale, Bin Yang, Gayathri Ethiraj, Jesse Mendez
Dharini Shree Venkatesan, Rinkle Rose Renny, Srinivas IB

05/07/2026

1 Summary

Reasoning models often commit to flawed strategies early in their chain-of-thought (CoT) and burn tokens executing them, even when their own internal representations suggest the approach is unpromising.

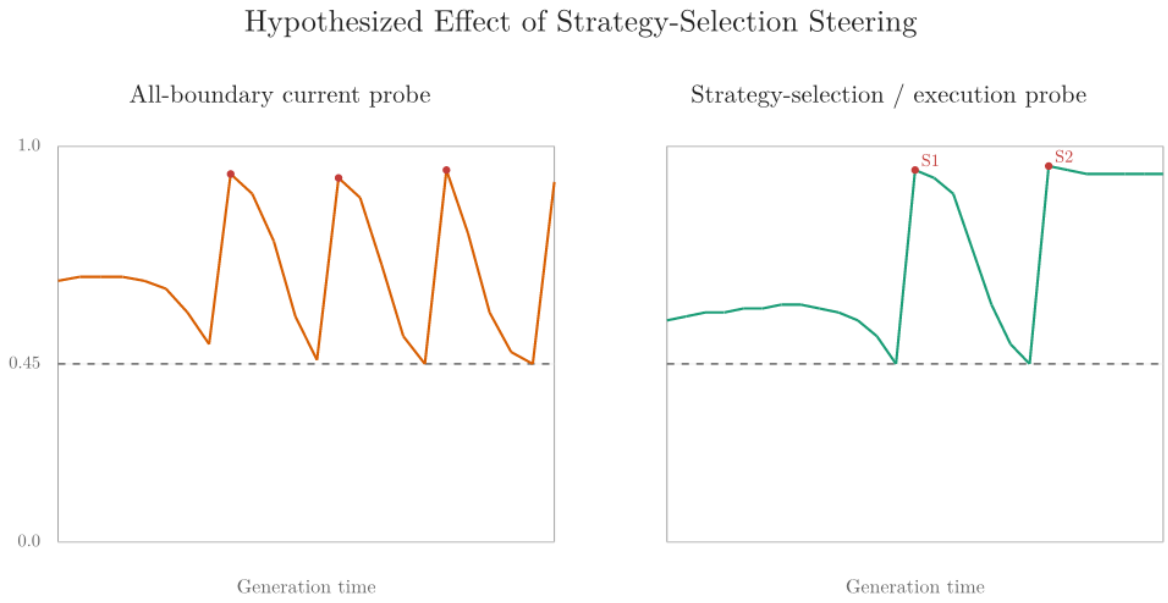
We propose a lightweight intervention framework that detects and corrects poor strategy choices in-flight, before the model wastes compute on doomed trajectories. Our system has two components. A **segmenter** classifies spans of a CoT trace as either *strategy-selection* (choosing or revisiting an approach) or *execution* (carrying out a plan), isolating the moments where intervention is most actionable.

At each strategy-selection point, a **probe**, a lightweight classifier trained on hidden states from ∼1000 AIME problems, estimates the probability that the current trajectory will yield a correct answer.

When probe confidence falls below threshold, we apply a steering vector to the last layer, computed as the mean difference between hidden states from successful and failed traces, nudging the model toward more promising reasoning directions without interrupting generation.

We evaluate the probe using ROC-AUC, Expected Calibration Error, and Brier score. We compare against an **all-boundary probe** so as to serve as a baseline for our central hypothesis: that *when* you intervene in a reasoning trace matters as much as *whether* you intervene at all.

We faced multiple challenges during this project, the largest of which was that multiple separate pieces built up to the magnum opus - the strategy selection/execution target probe - all of which had to have nearly perfect architectural decisions behind them so as to not pollute the rest of the steps. As such, the challenges here were difficult to overcome, and we didn't have the time to train the strategy selection/execution probe at all because of them.



2 Architectural Choices

A few architectural decisions touched all parts of the project:

- Model used for COT: DeepSeek-R1-Distill-Qwen-7B due to its long reasoning traces and extraordinary math abilities for such a small model. Choosing a distilled model also allowed us to cut down on compute.
- Dataset: gneubig/aime-1983-2024 for historical AIME problems.
- Paragraph Level Spans: throughout this project, we segment reasoning traces and extract hidden states at paragraph-level (`\n\n`) spans, following the convention adopted in recent reasoning-model literature (R^2MU, Wang et al. 2025; CREST, Zhang et al. 2025).
- We only saved hidden states at the last layer (28) throughout.

2.1 Collecting Hidden States over AIME traces

The idea here was two-pronged: set up the infrastructure to train a segmenter over all ∼950 CoT traces we generated with `DeepSeek-R1-Distill-Qwen-7B`, and produce labeled training data in the form of `(hidden-states-at-end-of-paragraph, 0/1 label for whether the problem was answered correctly)` for the probe.

It may help to see the structure of each problem we saved:

Field	Description
`hidden_states`	Layer-28 hidden states at paragraph breaks (`\n\n`) inside the CoT, capped at 100 per problem
`cot_trace`	Decoded tokens between `<think>...</think>`
`raw_output`	Full generation including CoT and final answer
`model_answer`	`math-verify`-parsed final answer (`<none>` if no parseable answer)
`label`	1 if `model_answer` matches the gold AIME answer, else 0
`truncated`	1 if generation hit `MAX_NEW_TOKENS`

Methodology

Distributed Collection: Generating 1000 traces on one person's compute budget was infeasible (∼5 hr on an H100 per 250 problems). We split the dataset into disjoint problem ranges across four teammates, each running a certain range with their own `START_IDX`/`END_IDX`.

Grading: Final answers are graded with `math-verify`, which is widely regarded as state of the art for grading, which parses LaTeX math and checks symbolic equivalence.

Truncation: A trace is flagged `truncated=1` if it hits `MAX_NEW_TOKENS` *and* the last token isn't EOS. Truncated traces are kept in the dataset with the flag set rather than dropped, since hard problems disproportionately truncate and dropping them would bias the probe toward easy problems.

Challenges

- We likely did not need to actually collect hidden states/COT traces for all ∼1000 AIME problems, and I would guess that only 50–100 traces were actually needed. We wasted a lot of time and compute gathering all of them.

2.2 Steering

For both the baseline probe and the strategy selection/execution probe, we probe the last-token hidden state, and if `P(correct) < 0.45` (empirically, this gave us a decent number of steers per problem) we add $\alpha \cdot v$ to that hidden state in place before it reaches the LM head, where $v = \text{mean}(h^+) - \text{mean}(h^-)$ is the global mean-difference vector over all collected activations and $\alpha = 0.8$ (this pushes confidence up $\sim$42 percentage points).

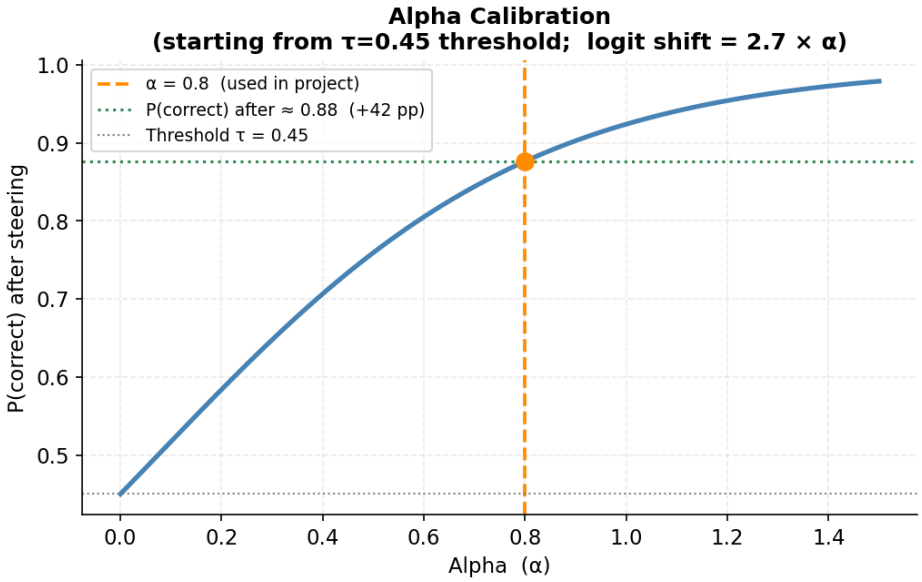
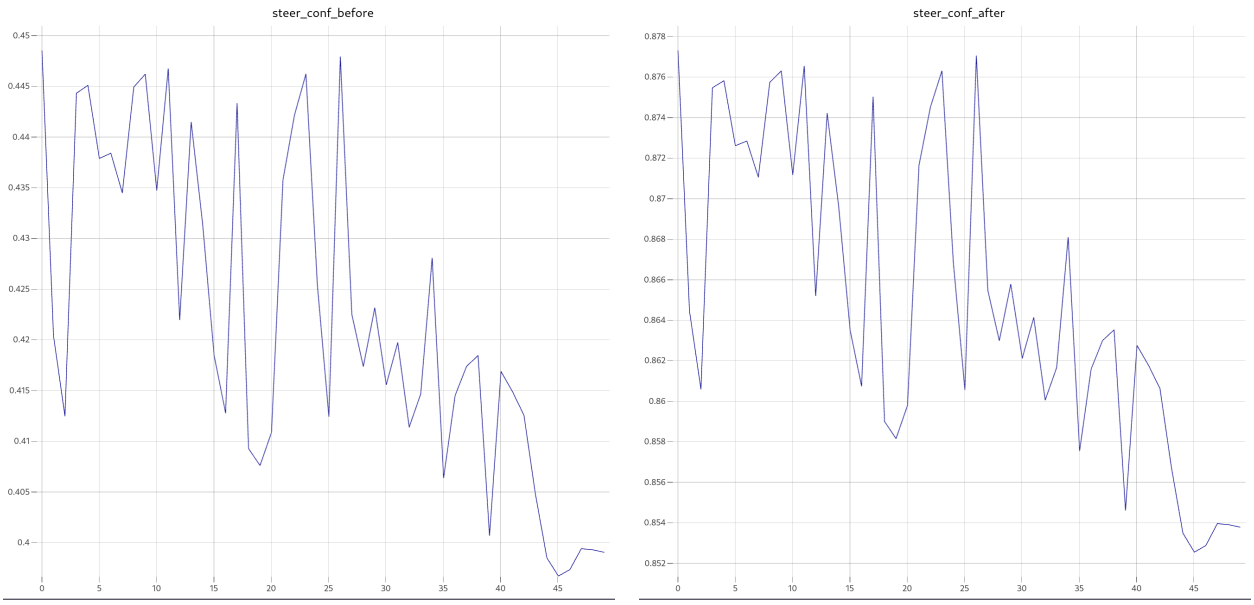


Figure 1: Alpha calibration

Because the probe is linear, post-steering probe confidence rises by construction and is therefore not a success signal, the only meaningful evaluation is final-answer correctness and whether or not it (a) achieved the same problems correct without steering as with steering, and (b) did *better* than without steering, and in the case of the target probe, did better than the baseline all-boundary probe, which simultaneously measures recovery (failures flipped to correct) and robustness (successes preserved).

Here is what we mean:



Notice the y-axis scales changing, and that even if we boost confidence rates by applying a forcing vector at the end of a paragraph, they still just continue to drop if the model hasn't found a better strategy.

2.3 All Boundary Baseline Probe

This probe is trained on *every* paragraph given to it, and it differs from our non-baseline strategy selection/execution probe in that the other probe will be given a *narrower* set of data after we classify paragraphs for strategy selection, which this probe does not take into account.

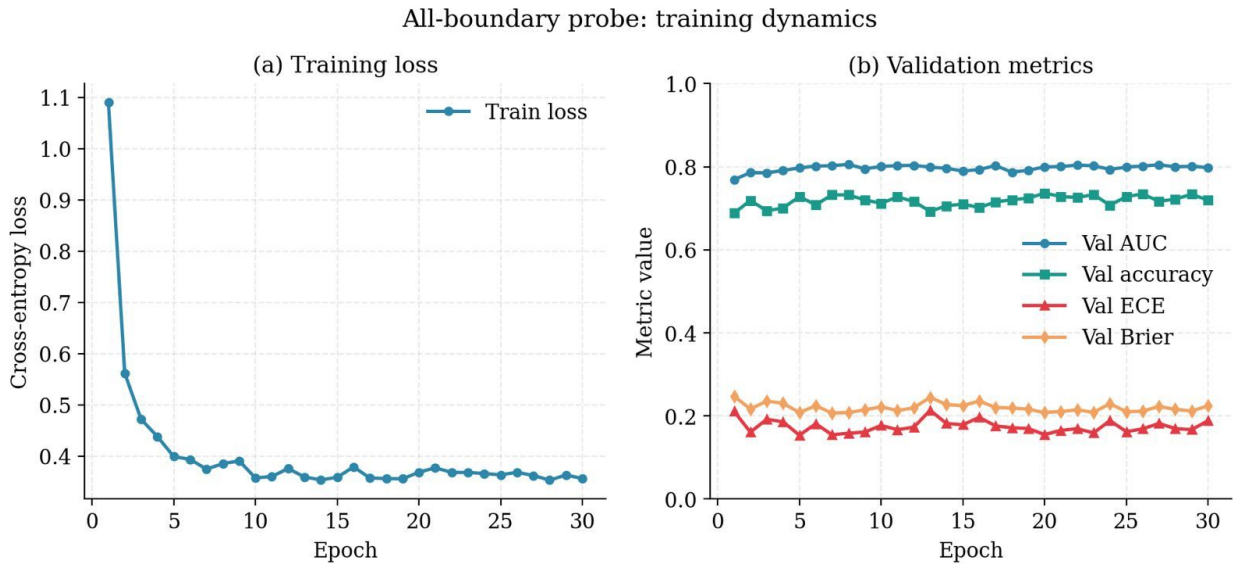
The hypothesis here is that where we steer – at strategy selection points, matters. If our strategy selection/execution probe outperforms this all-boundary baseline probe, we have proved our hypothesis.

Methodology

The fact that we saved hidden states at the end of each paragraph allowed us to train a probe to predict, given any hidden state at the last layer of this model, whether or not the model will output a correct answer (1) or wrong answer (0) as a probability between 0 and 1.

We ran normalization over the hidden states, as well as a 40-trial Optuna sweep for quite a few hyperparameters, partially because this was extremely cheap to train en masse and we wanted to take advantage of that fact.

The ideal hyperparameters turned out to be: `lr=2.3e-4, weight-decay=2.2e-3, batch=512, epochs=51`.



Challenges

- We chose only the last (28th) layer to train the all-boundary baseline probe on, so we couldn't properly nudge the middle-late layers because the scaling for α was all off, and it was predicting something else entirely.
- We didn't have time to sweep alpha as a hyperparameter; I arbitrarily set it at 0.8 because some math indicated it'd strongly push the model in a more confident direction.

Metrics

Note that we steered 50 problems total uniformly at random: 25 originally correct, 25 originally failed.

- The probe has a validation AUC of 0.8707
- Original accuracy: 25/50 (50.0%)
- Steered accuracy: 29/50 (58.0%)
- Originally-correct kept: 22/25; regressions: 3/25
- Originally-failed recovered: 7/25
- Steering hyperparameters: $\alpha = 0.8$, threshold $= 0.45$, max steers/problem $= 50$
- Total steering interventions: 1078; mean per problem: 21.56

Here, the most important things we saved were: what the label originally was for the problem before steering with the all-boundary probe, what the label was after, and how many tokens it used (for later comparison with the strategy selection/execution probe).

These allow us to accurately compare this baseline result against our strategy selection/execution probe in the future.

2.4 Strategy Selection/Execution Target Probe

Just to be perfectly clear, we didn't have a chance to set up the training for this at all, but here is broadly the methodology we'd use:

Methodology

The differentiating factor here versus our baseline probe is the *narrowing* of the (hidden states, label) dataset. That is, we only take hidden states from spans (paragraphs) classified as strategy selection points.

Broadly:

1. We take our massive set of traces, segment and classify all of them,
2. Take only the hidden states at the end of paragraphs that got classified as strategy selection points,
3. Construct this dataset of (hidden-states-at-strategy-selection-points, label of 0/1)
4. Train the probe on it with the same hyperparameters as in the all boundary baseline probe.

2.5 Segmenter

Pipeline

DeepSeek-R1-Distill-Qwen-7B generates CoT traces with hidden states → split the CoT into different spans based on the paragraph cutoff → the segmenter (LLM-as-a-judge) labels each span as strategy selection or execution → saves it to a JSON file.

Strategy: The model is choosing, planning, or deciding what approach to take. Examples include:

- Problem framing: "I need to find...", "Let me understand..."
- Approach selection: "Let me try using AM-GM", "Maybe I can substitute..."
- Reconsidering: "Wait, that doesn't seem right", "Hmm, let me think again"
- Comparing options: "Alternatively...", "Another approach would be..."
- Key insights: "The trick here is...", "This is a key observation"

Execution: The model is carrying out the chosen plan from the Strategy. Examples include:

- Algebraic manipulation: expanding, factoring, simplifying
- Substitution: plugging values into formulas
- Computation: arithmetic, evaluating expressions
- Applying formulas: quadratic formula, binomial theorem
- Stating results: "Therefore...", "Thus...", "The answer is..."
- Verification by calculation: re-computing to check

A simplified way to do this would've been coming up with a set of keywords the model consistently uses (empirically) within strategy selection paragraphs, but LLMs give us more flexibility, and we noticed most frontier models do about as well as humans at this task, since there's no real "ground truth" here.

Challenges

- Our first approach was to use the same reason model that generated the CoT (DeepSeek-R1-Distill-Qwen-7B) as the judge, which failed completely where it tried to solve the problem and returned the labels as EXECUTION. Later switch to instruction model Qwen3.5 27B, but the problem it takes a very long time to label and consumes a lot of compute resources. Then, we switched to using Claude Haiku 4.5 through the API.
- We didn't have time to train a classifier over whether or not paragraph spans were strategy selection or execution, and instead just used an LLM as a judge, with a very minute amount of human-annotated spans to back it up.

3 Limitations

- **Single model, single domain.** All experiments use DeepSeek-R1-Distill-Qwen-7B on AIME problems. Findings may not transfer to other reasoning models or to non-mathematical domains.
- **Mid-layer probing.** Hidden states were collected exclusively at layer 28. Prior work shows correctness signals vary substantially across layers; our layer choice may not be optimal for all reasoning types.
- **Greedy decoding.** Generation is deterministic (do_sample=False), so we cannot assess whether the intervention is robust under sampling temperature variation.
- **Small steering evaluation set.** The 50-problem evaluation (25 originally correct, 25 originally failed) is too small for statistically significant claims; the observed +8 pp accuracy gain on paired evaluation has wide confidence intervals.
- **Strategy probe not trained.** The central artifact of the project, the strategy-selection-conditioned probe, was not trained due to time constraints, so the comparison against the all-boundary baseline that motivates our hypothesis is left to future work.
- **LLM-as-judge for segmentation.** Span-level strategy/execution labels come from Claude Haiku 4.5 with limited human verification. Without a held-out human-annotated set the segmenter's error rate is unknown.
- **Unfaithful chain-of-thought.** As shown by prior work, CoT text can diverge from the underlying computation. Our segmenter operates on text, while our probe operates on hidden states; misalignment between these two views is possible.

4 Future Work

If we had more compute and time, several directions would strengthen our work.

Collecting Hidden States over AIME traces

- Take a few middle-late layers instead of just the last (28th) layer.
- Mimic a strategy similar to CREST, Zhang et al. 2025.

Strategy Selection/Execution Target Probe

- We'd train this using the methodology outlined above!
- Furthermore, given infinite time and compute, we'd try steering and getting hidden states from other layers in COT traces

Segmenter

- First, we would hand-annotate quite a few more paragraph spans across the dataset with multiple annotators, and use that to compare the LLM-as-a-judge against.
- Second, we would scale up the local instruction model from Qwen 3.5 27B to a higher 70B+ model, which would close the quality gap with API-based Labeling while keeping everything local and reproducible.
- Third, we would do a systematic cross-method comparison, such that we take the human annotation labels as the gold standard, then measure accuracy for each method, eg. frontier out-of-the-box model, local model trained on human annotations, keyword-based, paragraph split-based heuristics, sentence split-based heuristics etc.

References

[1] Zhenyu Zhang et al. Understanding and Steering the Cognitive Behaviors of Reasoning Models at Test-Time. *arXiv preprint arXiv:2512.24574*, 2025. https://arxiv.org/abs/2512.24574

[2] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning Models Know When They're Right: Probing Hidden States for Self-Verification. *arXiv preprint arXiv:2504.05419*, 2025. https://arxiv.org/abs/2504.05419

[3] Amos Azaria and Tom Mitchell. The Internal State of an LLM Knows When It's Lying. *arXiv preprint arXiv:2304.13734*, 2023. https://arxiv.org/abs/2304.13734

[4] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering Language Models With Activation Engineering. *arXiv preprint arXiv:2308.10248*, 2023. https://arxiv.org/abs/2308.10248

[5] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025. https://arxiv.org/abs/2501.12948

[6] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021. https://arxiv.org/abs/2103.03874

[7] Hemish Veeraboina. AIME Problem Set 1983–2024. *Kaggle / Hugging Face dataset*, 2023. https://huggingface.co/datasets/gneubig/aime-1983-2024

[8] Jason Zhang and Scott Viteri. Uncovering Latent Chain of Thought Vectors in Language Models. *arXiv preprint arXiv:2409.14026*, 2024. https://arxiv.org/abs/2409.14026